

ToolRuntime

It is a specialized class in LangChain for dependency injection that gives tools direct access to the application's runtime context (like the current graph state, user context, memory store, and more) without exposing these backend properties to LLMs.

`runtime` object containing context (static info like `userID`, DB connections), a store (for long-term memory), a stream writer (for streaming progress), and execution info.

When a tool function has a parameter named `runtime` with the type hint `ToolRuntime`, the tool execution system automatically injects an instance containing:

- `state` — the current graph state
- `tool_call_id` — the ID of the current tool call
- `config` — the `RunnableConfig` for the current execution
- `context` — runtime context (e.g., user info)
- `store` — a `BaseStore` for long-term memory

```
from langchain_core.tools import tool
from langchain.tools import ToolRuntime
from dataclasses import dataclass

@dataclass
class Context:
    user_name: str

@tool
def fetch_user_name(runtime: ToolRuntime) -> str:
    """Fetches the current user's name."""
    return f"Hello, {runtime.context.user_name}!"
```

Accessing different parts

```
@tool
def my_tool(x: int, runtime: ToolRuntime) -> str:
    """Tool that accesses runtime context."""

    # Current graph state
    messages = runtime.state["messages"]

    # Who triggered this tool call
    print(f"Tool call ID: {runtime.tool_call_id}")

    # User-specific context
    user_id = runtime.context.get("user_id")
```

```
# Long-term memory store
runtime.store.put(("user_prefs",), user_id, {"theme": "dark"})

return "done"
```